Lab 8 - Aggregations - Pipelines

2.1 - Resumen de transacciones por cuenta

```
db.transactions.aggregate([
  {
    $unwind: "$transactions",
  },
  {
    $group: {
      _id: "$account_id",
      total_transactions: { $sum: 1 },
      total_amount: { $sum: "$transactions.amount" },
      average_amount: { $avg: "$transactions.amount" },
    },
  },
  // buscar con customers
  {
    $lookup: {
      from: "customers",
      localField: "_id",
      foreignField: "accounts",
      as: "customer_info",
    },
  },
  {
    $project: {
      account_id: "$_id",
      name: { $arrayElemAt: ["$customer_info.name", 0] },
      city: {
        $let: {
          vars: {
            address: { $arrayElemAt: ["$customer_info.address", 0] },
            parts: {
              $split: [{ $arrayElemAt: ["$customer_info.address", 0] }, "\n"],
            },
          },
          in: { $arrayElemAt: ["$$parts", 1] },
        },
```

```
      },
      total_transactions: 1,
      average_amount: { $round: ["$average_amount", 2] },
      _id: 0,
    },
  },
  {
    $sort: { total_transactions: -1 },
  },
]);
```

Resultado

```
{
  total_transactions: 100,
  account_id: 675420,
  name: 'Vincent Rose',
  city: 'Candicehaven, KY 31841',
  average_amount: 5161.05
}
{
  total_transactions: 100,
  account_id: 632807,
  name: 'John Williams',
  city: 'FPO AE 20182',
  average_amount: 4965.82
}
{
  total_transactions: 100,
```

Figure 1: Resultado Ejecución Pipeline

2.2 – Clasificación de cuentas por balance

```
db.accounts.aggregate([
  {
    $lookup: {
      from: "account_summaries",
      localField: "products",
      foreignField: "_id",
      as: "balance_info",
    },
  },
```

```
{
  $unwind: "$balance_info",
},
{
  $group: {
    _id: "$account_id",
    total_balance: { $sum: { $toDouble: "$balance_info.average_balance" } },
    account_data: { $first: "$$ROOT" },
  },
},
{
  $lookup: {
    from: "customers",
    localField: "account_data.account_id",
    foreignField: "accounts",
    as: "customer_info",
  },
},
{
  $unwind: "$customer_info",
},
{
  $project: {
    name: "$customer_info.name",
    total_balance: 1,
    category: {
      $switch: {
        branches: [
          { case: { $lt: ["$total_balance", 5000] }, then: "Bajo" },
          {
            case: {
              $and: [
                { $gte: ["$total_balance", 5000] },
                { $lte: ["$total_balance", 20000] },
              ],
            },
            then: "Medio",
          },
          { case: { $gt: ["$total_balance", 20000] }, then: "Alto" },
```

```
          ],
          default: "Sin categoría",
        },
      },
    },
  },
  {
    $sort: { total_balance: -1 },
  },
]);
```

Resultado

```
{
  _id: 627788,
  total_balance: 162974207.0097825,
  name: 'Ashley Rodriguez',
  category: 'Alto'
}
  _id: 627788,
  total_balance: 162974207.0097825,
  name: 'Shawn Austin',
  category: 'Alto'
}
{
  _id: 300446,
```

Figure 2: Resultado Ejecución Pipeline

2.3 – Balance máximo por ciudad

```
db.accounts.aggregate([
  {
    // resumen para balance
    $lookup: {
      from: "account_summaries",
      localField: "products",
      foreignField: "_id",
      as: "balance_data",
    },
```

```
},
{ $unwind: "$balance_data" },
{
  $group: {
    // balance total
    _id: "$account_id",
    total_balance: { $sum: { $toDouble: "$balance_data.average_balance" } },
    original_doc: { $first: "$$ROOT" },
  },
},
{
  // info clientes
  $lookup: {
    from: "customers",
    localField: "_id",
    foreignField: "accounts",
    as: "customer_data",
  },
},
{
  // solo asociados
  $match: { customer_data: { $ne: [] } },
},
{ $unwind: "$customer_data" },
{
  // ciudad
  $addFields: {
    city: {
      $let: {
        vars: {
          addressParts: { $split: ["$customer_data.address", "\n"] },
        },
        in: { $arrayElemAt: ["$$addressParts", 1] },
      },
    },
  },
},
{ $sort: { city: 1, total_balance: -1 } },
{
```

```
    // agrupar por ciudad y max balance mayor
    $group: {
      _id: "$city",
      name: { $first: "$customer_data.name" },
      max_balance: { $first: "$total_balance" },
    },
  },
  {
    $project: {
      _id: 0,
      ciudad: "$_id",
      nombre: "$name",
      balance_total: "$max_balance",
    },
  },
  {
    // ordenar ciudad alfabéticamente
    $sort: { ciudad: 1 },
  },
]);
```

Resultado

```
{
  ciudad: 'APO AA 21293',
  nombre: 'Bradley Lopez',
  balance_total: 80327484.20022263
}
{
  ciudad: 'APO AA 32725',
  nombre: 'Sandra Armstrong',
  balance_total: 80095508.15518327
}
{
  ciudad: 'APO AA 46452',
  nombre: 'James Frazier',
```

Figure 3: Resultado Ejecución Pipeline

2.4 – Top 10 transacciones recientes y grandes

```
db.transactions.aggregate([
  { $unwind: "$transactions" },
  // filtro 6 meses
  {
    $match: {
      "transactions.date": {
        $gte: new Date("2016-06-01"),
        $lte: new Date("2016-12-31"),
      },
    },
  },
  { $sort: { "transactions.amount": -1 } },
  { $limit: 10 },
  {
    $lookup: {
      from: "accounts",
      localField: "account_id",
      foreignField: "account_id",
      as: "account_info",
    },
  },
  { $unwind: "$account_info" },
  // info clientes
  {
    $lookup: {
      from: "customers",
      localField: "account_info.account_id",
      foreignField: "accounts",
      as: "customer_info",
    },
  },
  { $unwind: "$customer_info" },
  {
    $project: {
      _id: 0,
      fecha_transaccion: "$transactions.date",
      monto: "$transactions.amount",
```

```
      tipo: "$transactions.transaction_code",
      simbolo: "$transactions.symbol",
      cliente: "$customer_info.name",
      ciudad: {
        $arrayElemAt: [{ $split: ["$customer_info.address", "\n"] }, 1],
      },
      cuenta_id: "$account_id",
    },
  },
]);
```

Resultado

```
{
  fecha_transaccion: 2016-10-28T00:00:00.000Z,
  monto: 9999,
  tipo: 'buy',
  simbolo: 'fb',
  cliente: 'Annette Watts',
  ciudad: 'North Loriside, IN 96979',
  cuenta_id: 107787
}
{
  fecha_transaccion: 2016-08-02T00:00:00.000Z,
  monto: 9996,
  tipo: 'buy',
  simbolo: 'team',
  cliente: 'Anna Flowers',
  ciudad: 'Castilloport, KY 49162',
  cuenta_id: 972116
}
```

Figure 4: Resultado Ejecución Pipeline

2.5 – Variación porcentual entre transacciones más antigua y más reciente por cliente

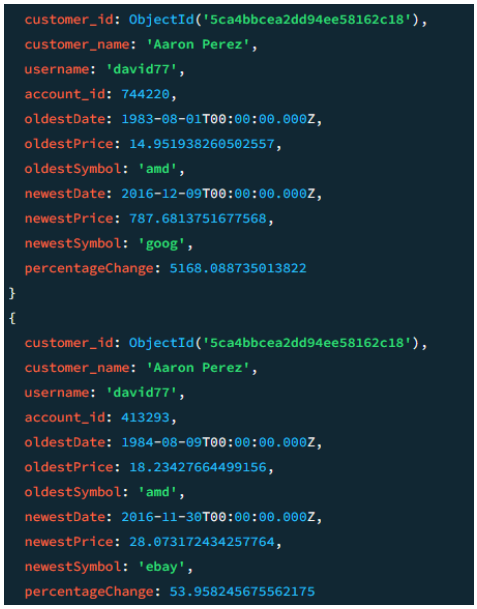
```
db.customers.aggregate([

  { $unwind: "$accounts" },


  { $lookup: {
      from: "transactions",
      localField: "accounts",
```

```
    foreignField: "account_id",
    as: "accountTransactions"
  }
},


{ $match: { "accountTransactions": { $ne: [] } } },


{ $unwind: "$accountTransactions" },


{ $unwind: "$accountTransactions.transactions" },


{ $group: {
    _id: {
      customer_id: "$_id",
      customer_name: "$name",
      username: "$username",
      account_id: "$accounts"
    },
    transactions: { $push: "$accountTransactions.transactions" },
    oldestTransaction: { $min: "$accountTransactions.transactions.date" },
    newestTransaction: { $max: "$accountTransactions.transactions.date" }
  }
},


{ $project: {
    _id: 0,
    customer_id: "$_id.customer_id",
    customer_name: "$_id.customer_name",
    username: "$_id.username",
    account_id: "$_id.account_id",
    transactions: 1,
    oldestTransaction: 1,
    newestTransaction: 1
  }
```

```
},


{ $addFields: {
    oldestTransactionData: {
      $filter: {
        input: "$transactions",
        as: "t",
        cond: { $eq: ["$$t.date", "$oldestTransaction"] }
      }
    },
    newestTransactionData: {
      $filter: {
        input: "$transactions",
        as: "t",
        cond: { $eq: ["$$t.date", "$newestTransaction"] }
      }
    }
  }
},


{ $project: {
    customer_id: 1,
    customer_name: 1,
    username: 1,
    account_id: 1,
    oldestDate: "$oldestTransaction",
    oldestPrice: { $toDouble: { $arrayElemAt: ["$oldestTransactionData.price",
     ↪  0] } },
    oldestSymbol: { $arrayElemAt: ["$oldestTransactionData.symbol", 0] },
    newestDate: "$newestTransaction",
    newestPrice: { $toDouble: { $arrayElemAt: ["$newestTransactionData.price",
     ↪  0] } },
    newestSymbol: { $arrayElemAt: ["$newestTransactionData.symbol", 0] },
    percentageChange: {
      $multiply: [
        { $divide: [
            { $subtract: [
```

```
                { $toDouble: { $arrayElemAt: ["$newestTransactionData.price", 0]
↪  } },
                { $toDouble: { $arrayElemAt: ["$oldestTransactionData.price", 0]
↪  } }
              ]},
            { $toDouble: { $arrayElemAt: ["$oldestTransactionData.price", 0] } }
            ]
          },
          100
        ]
      }
    }
  },


  { $sort: { "customer_name": 1 } }
])
```

Resultado



```
  customer_id: ObjectId('5ca4bbcea2dd94ee58162c18'),
  customer_name: 'Aaron Perez',
  username: 'david77',
  account_id: 744220,
  oldestDate: 1983-08-01T00:00:00.000Z,
  oldestPrice: 14.951938260502557,
  oldestSymbol: 'amd',
  newestDate: 2016-12-09T00:00:00.000Z,
  newestPrice: 787.6813751677568,
  newestSymbol: 'goog',
  percentageChange: 5168.088735013822
}
{
  customer_id: ObjectId('5ca4bbcea2dd94ee58162c18'),
  customer_name: 'Aaron Perez',
  username: 'david77',
  account_id: 413293,
  oldestDate: 1984-08-09T00:00:00.000Z,
  oldestPrice: 18.23427664499156,
  oldestSymbol: 'amd',
  newestDate: 2016-11-30T00:00:00.000Z,
  newestPrice: 28.073172434257764,
  newestSymbol: 'ebay',
  percentageChange: 53.958245675562175
```

Figure 5: Resultado Ejecución Pipeline

2.6 – Agrupación de transacciones por mes y tipo con totales y promedios

```
db.transactions.aggregate([

  { $unwind: "$transactions" },



  {
    $addFields: {
      month: {
        $dateToString: { format: "%m", date: "$transactions.date" }
      }
    }
  },



  {
    $group: {
      _id: {
        month: "$month",
        transaction_code: "$transactions.transaction_code"
      },
      totalAmount: { $sum: "$transactions.amount" },
      avgAmount: { $avg: "$transactions.amount" },
      totalValue: { $sum: { $toDouble: "$transactions.total" } },
      avgValue: { $avg: { $toDouble: "$transactions.total" } }
    }
  },



  {
    $sort: { "_id.month": 1, "_id.transaction_code": 1 }
  }
]);
```

```
< {
  _id: {
    month: '01',
    transaction_code: 'buy'
  },
  totalAmount: 17852136,
  avgAmount: 5004.804037005887,
  totalValue: 1338810960.6186101,
  avgValue: 375332.4812499608
}
{
  _id: {
    month: '01',
    transaction_code: 'sell'
  },
  totalAmount: 17308451,
  avgAmount: 5024.223802612481,
  totalValue: 1277768414.860446,
  avgValue: 370905.20024976664
}
```


Figure 6: Resultado Ejecución Pipeline


```
  _id: {
    month: '02',
    transaction_code: 'sell'
  },
  totalAmount: 16052166,
  avgAmount: 5016.301875,
  totalValue: 11113573953.480551,
  avgValue: 347991.8604626722
}
{
  _id: {
    month: '03',
    transaction_code: 'buy'
  },
  totalAmount: 18348817,
  avgAmount: 5013.337978142076,
  totalValue: 1294678720.6449924,
  avgValue: 353737.35536748427
}
```


Figure 7: Resultado Ejecución Pipeline


```
{
  _id: {
    month: '03',
    transaction_code: 'sell'
  },
  totalAmount: 18251822,
  avgAmount: 4986.836612821858,
  totalValue: 1241345667.3771837,
  avgValue: 339165.4828899409
}
{
  _id: {
    month: '04',
    transaction_code: 'buy'
  },
  totalAmount: 17168890,
  avgAmount: 4912.414878397711,
  totalValue: 1294818139.7305083,
  avgValue: 370477.29319900094
}
```


Figure 8: Resultado Ejecución Pipeline

Resultado

2.7 – Identificación y almacenamiento de clientes inactivos

Se uso el siguiente script de agregation

```
db.customers.aggregate([
  {
    $lookup: {
      from: "transactions",
      localField: "accounts",
      foreignField: "account_id",
      as: "transactions"
    }
  },
  {
    $project: {
      _id: 1,
      username: 1,
      name: 1,
      email: 1,
      accounts: 1,
      active: 1,

      inactive_accounts: {
        $setDifference: ["$accounts", { $map: { input: "$transactions", as: "t",
         ↪  in: "$$t.account_id" } }]
      }
    }
  },
  {
    $match: {
      "inactive_accounts": { $ne: [] }
    }
  },
  {
    $project: {
      "_id": 1,
      "username": 1,
      "name": 1,
```

```
      "email": 1,
      "accounts": 1,
      "active": 1
    }
  },
  {
    $out: "inactive_customers"
  }
]);
```

Resultado

Devolvio como resultado un [] vacio de customers asi que se probo agregando un account nuevo sin ninguna transaccion y agregandolo en el customer lo que devolvio fue algo como

Figure 9: alt text

Y aqui la evidencia que se guardo en una coleccion nueva

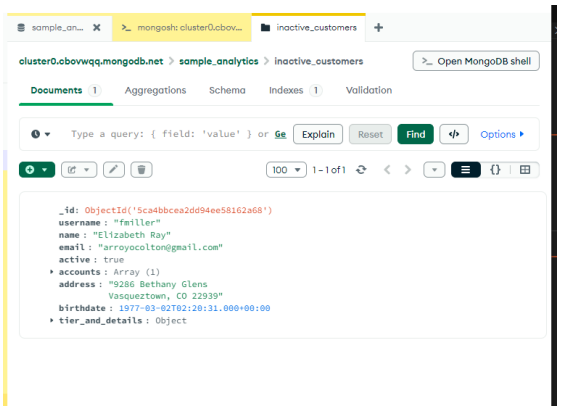


Figure 10: alt text

2.8 – Pipeline de agregación para crear un resumen de cuentas

Estrategia:

1. **Descomponer el array de productos en documentos individuales.**
   Se utiliza $unwind para dividir el arreglo products en documentos individuales. Esto permite trabajar cada tipo de producto (cuenta) por separado.

2. **Hacer un lookup para obtener las transacciones de cada cuenta.**
   Se aplica $lookup para hacer un "join" con la colección transactions. La unión se realiza usando el campo account_id, y las transacciones relacionadas se guardan en el campo account_transactions, que es un arreglo de documentos.

3. **Calcular el balance de cada cuenta.**
   Se usa $addFields para agregar un nuevo campo llamado balance, el cual representa la suma de los totales de todas las transacciones asociadas a la cuenta.

   - **balance:** Se crea como un nuevo campo temporal.
   - **$map:** Recorre el arreglo de transacciones para transformar cada una.
   - **input:** Se toma el arreglo account_transactions.transactions, accediendo al primer elemento con $arrayElemAt (["$account_transactions.transactions", 0]), que representa la lista de transacciones de la cuenta.
   - **as:** Cada transacción individual se referencia como t.
   - **in:** Para cada t, se extrae el campo total y se convierte a número decimal usando $toDecimal, ya que originalmente es un string.
   - **$sum:** Se suman todos los valores transformados para obtener el balance de la cuenta.

4. **Agrupar por producto y calcular el total de cuentas y el balance promedio.**
   Se aplica $group para agrupar los documentos por tipo de producto (products). En este paso se calcula:

   - total_accounts: El número total de cuentas por tipo de producto.
   - average_balance: El promedio de balance de las cuentas agrupadas por producto.

5. **Guardar el resultado en la colección account_summaries.**
   Finalmente, se utiliza $merge para guardar el resultado en la colección account_summaries. Esta operación inserta nuevos documentos o actualiza los existentes, según corresponda.

```
[
  {
    $unwind:
```

```
    {
      path: "$products",
    },
  },
  {
    $lookup:
      {
        from: "transactions",
        localField: "account_id",
        foreignField: "account_id",
        as: "account_transactions",
      },
  },
  {
    $addFields:
      {
        balance: {
          $sum: {
            $map: {
              input: {
                $arrayElemAt: ["$account_transactions.transactions", 0],
              },
              as: "t",
              in: {
                $toDecimal: "$$t.total",
              },
            },
          },
        },
      },
  },
  {
    $group:
      {
        _id: "$products",
        total_accounts: {
          $sum: 1,
        },
        average_balance: {
```

```
          $avg: "$balance",
        },
      },
  },
  {
    $merge:
      {
        into: "account_summaries",
      },
  },
]
```

Resultado

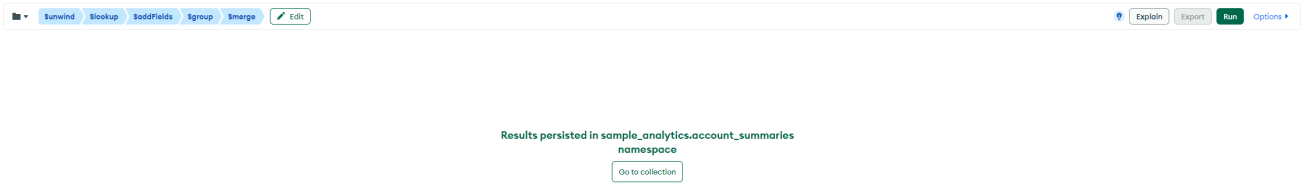


Figure 11: Resultado Ejecución Pipeline

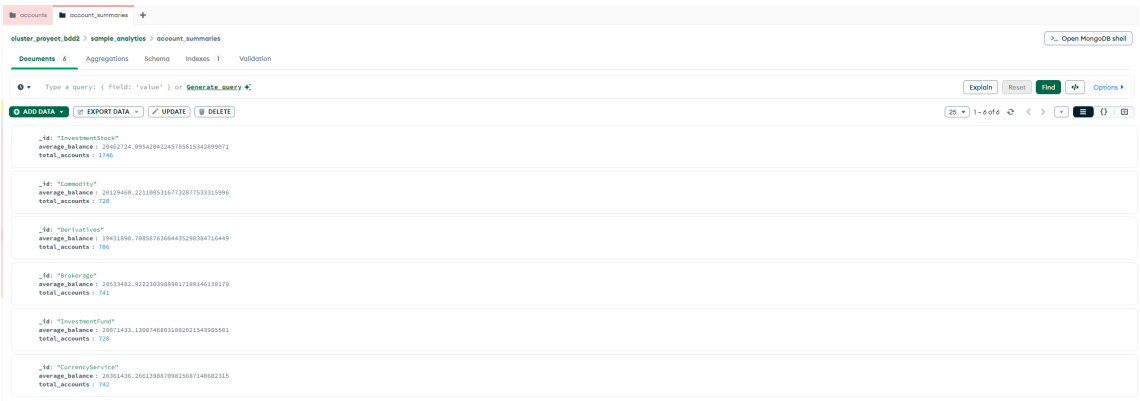


Figure 12: Resultado Account Summaries

2.9 – Pipeline de agregación para identificar clientes de alto valor

Estrategia:

1. **Hacer un lookup para obtener las transacciones de cada cliente.**
   Se utiliza $lookup para realizar un "join" entre la colección actual (clientes) y la colección transactions.
   - **localField:** accounts, que contiene los account_id asociados al cliente.

- **foreignField:** account_id, el campo en la colección transactions.
- **as:** Se almacena el resultado en el nuevo campo customer_transactions, que será un arreglo de documentos con las transacciones de cada cuenta.

2. **Calcular el balance total y el número de transacciones por cliente.**
   Se aplica $addFields para agregar dos nuevos campos:
   - **total_balance:**
     Se calcula sumando los montos de todas las transacciones del cliente.
     - **$map:** Recorre el arreglo customer_transactions.
     - **input:** Cada elemento del arreglo representa un grupo de transacciones por cuenta.
     - **as:** Se referencia a cada grupo como trans.
     - **in:**
       Para cada grupo trans, se suman los valores de total de sus transacciones individuales:
       * Se accede a trans.transactions.
       * Se usa otro $map para convertir cada total (originalmente string) a decimal usando $toDecimal.
       * Se suman los montos de las transacciones del grupo.
   - **total_transactions:**
     Se calcula sumando el campo transaction_count de cada elemento en customer_transactions, representando el total de transacciones del cliente.
3. **Filtrar solo clientes de alto valor.**
   Se utiliza $match para seleccionar únicamente los clientes que cumplen ambos criterios:
   - **total_balance > 30,000** unidades monetarias.
   - **total_transactions > 5** transacciones.
4. **Proyectar solo los campos relevantes.**
   Con $project, se seleccionan los campos que se quieren conservar en el resultado:
   - name
   - email
   - total_balance
   - total_transactions
5. **Guardar el resultado en la colección high_value_customers.**
   Finalmente, se usa $merge para guardar los resultados en la colección high_value_customers.
   Esta operación insertará nuevos documentos o actualizará los existentes si es necesario.

```
[
  {
    $lookup: {
      from: "transactions",
      localField: "accounts",
      foreignField: "account_id",
      as: "customer_transactions",
    },
  },
  {
    $addFields: {
      total_balance: {
        $sum: {
          $map: {
            input: "$customer_transactions",
            as: "trans",
            in: {
              $sum: {
                $map: {
                  input: "$$trans.transactions",
                  as: "t",
                  in: {
                    $toDecimal: "$$t.total",
                  },
                },
              },
            },
          },
        },
      },
      total_transactions: {
        $sum: "$customer_transactions.transaction_count",
      },
    },
  },
  {
    $match: {
      total_balance: {
        $gt: 30000,
```

```
      },
      total_transactions: {
        $gt: 5,
      },
    },
  },
  {
    $project: {
      name: 1,
      email: 1,
      total_balance: 1,
      total_transactions: 1,
    },
  },
  {
    $merge: {
      into: "high_value_customers",
    },
  },
]
```

Resultado

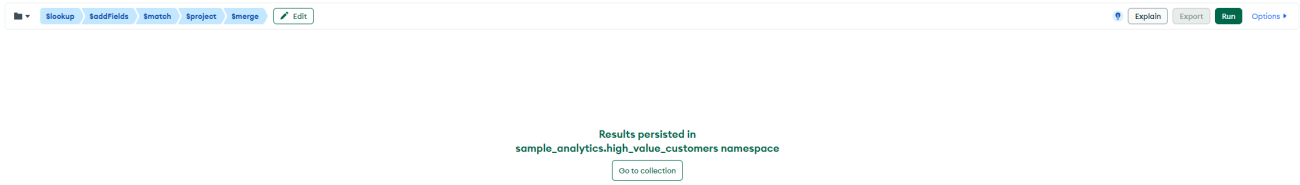


Figure 13: Resultado Ejecución Pipeline

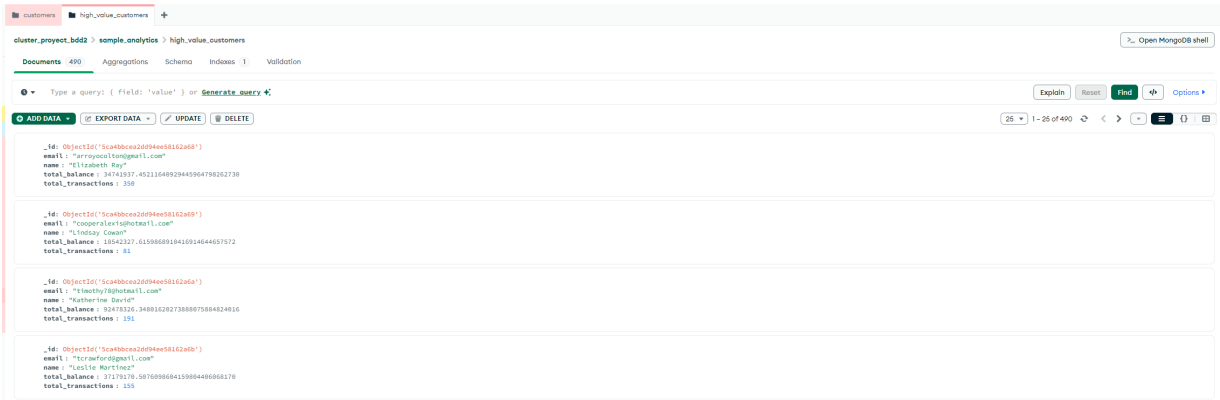


Figure 14: Resultado High Value Customers

2.10 – Clasificación de clientes según promedio mensual de transacciones en el último año

Estrategia:

1. **Relacionar clientes con sus transacciones.**
   Se usa $lookup para unir cada cliente con sus transacciones en la colección transactions.
   La unión se realiza usando el campo accounts del cliente y account_id de la transacción. El resultado se guarda en el campo customer_transactions.

2. **Desenrollar el array de transacciones.**
   Se aplican dos $unwind:
   - Primero en customer_transactions para separar cada conjunto de transacciones por cuenta.
   - Luego en customer_transactions.transactions para separar cada transacción individualmente.

3. **Proyectar los campos relevantes.**
   Se usa $project para quedarse únicamente con los datos importantes:
   - name del cliente.
   - transaction_date (fecha de cada transacción).
   - transaction_amount (monto de cada transacción).
   - year y month derivados de transaction_date.

4. **Agrupar la información por cliente.**
   Con $group se agrupa todo por cliente (_id del cliente):
   - Se guarda el primer name.
   - Se acumulan todas las transacciones en all_transactions.
   - Se guarda el conjunto de años (years) donde tuvo actividad.

5. **Ordenar las transacciones internamente.**
   Se usa $sortArray en $project para ordenar el array all_transactions por transaction_date de forma descendente.

6. **Identificar el último año de actividad.**
   Se calcula el last_transaction_year usando $max sobre el arreglo years.

7. **Filtrar solo las transacciones del último año.**
   Con $addFields se usa $filter para dejar únicamente las transacciones cuyo year sea igual a last_transaction_year.

8. **Proyectar los campos finales.**

Se usa $project para preparar los datos de salida:

- name.
- last_transaction_year.
- transactions (solo transaction_date y transaction_amount).
- count_transactions (número total de transacciones de ese año).
- transactions_average_year_month (promedio de transacciones por mes, dividiendo entre 12).

9. **Categorizar la frecuencia de las transacciones.**
   Finalmente con $addFields y $switch se asigna una categoría basada en el promedio mensual:

   - "infrequent" si el promedio es menor a 2.
   - "regular" si el promedio está entre 2 y 5.
   - "frequent" si el promedio es mayor a 5.

```
[
  {
    $lookup: {
      from: "transactions",
      localField: "accounts",
      foreignField: "account_id",
      as: "customer_transactions"
    }
  },
  {
    $unwind: "$customer_transactions"
  },
  {
    $unwind: "$customer_transactions.transactions"
  },
  {
    $project: {
      name: 1,
      transaction_date:
        "$customer_transactions.transactions.date",
      transaction_amount:
        "$customer_transactions.transactions.amount",
      year: {
        $year:
```

```
        "$customer_transactions.transactions.date"
    },
    month: {
      $month:
        "$customer_transactions.transactions.date"
    }
  }
},
{
  $group: {
    _id: "$_id",
    name: {
      $first: "$name"
    },
    all_transactions: {
      $push: {
        transaction_date: "$transaction_date",
        transaction_amount:
          "$transaction_amount",
        year: "$year",
        month: "$month"
      }
    },
    years: {
      $addToSet: "$year"
    }
  }
},
{
  $project: {
    name: 1,
    all_transactions: {
      $sortArray: {
        input: "$all_transactions",
        sortBy: {
          transaction_date: -1
        }
      }
    },
```

```
    last_transaction_year: {
      $max: "$years"
    }
  }
},
{
  $addFields: {
    transactions: {
      $filter: {
        input: "$all_transactions",
        as: "tran",
        cond: {
          $eq: [
            "$$tran.year",
            "$last_transaction_year"
          ]
        }
      }
    }
  }
},
{
  $project: {
    name: 1,
    last_transaction_year: 1,
    transactions: {
      transaction_date: 1,
      transaction_amount: 1
    },
    count_transactions: {
      $size: "$transactions"
    },
    transactions_average_year_month: {
      $divide: [
        {
          $size: "$transactions"
        },
        12
      ]
```

```
      }
    }
  },
  {
    $addFields: {
      transaction_frequency_category: {
        $switch: {
          branches: [
            {
              case: {
                $lt: [
                  "$transactions_average_year_month",
                  2
                ]
              },
              then: "infrequent"
            },
            {
              case: {
                $and: [
                  {
                    $gte: [
                      "$transactions_average_year_month",
                      2
                    ]
                  },
                  {
                    $lte: [
                      "$transactions_average_year_month",
                      5
                    ]
                  }
                ]
              },
              then: "regular"
            },
            {
              case: {
                $gt: [
```

```
                "$transactions_average_year_month",
                5
              ]
            },
            then: "frequent"
          }
        ],
        default: "unknown"
      }
    }
  }
}
]
```

Resultado

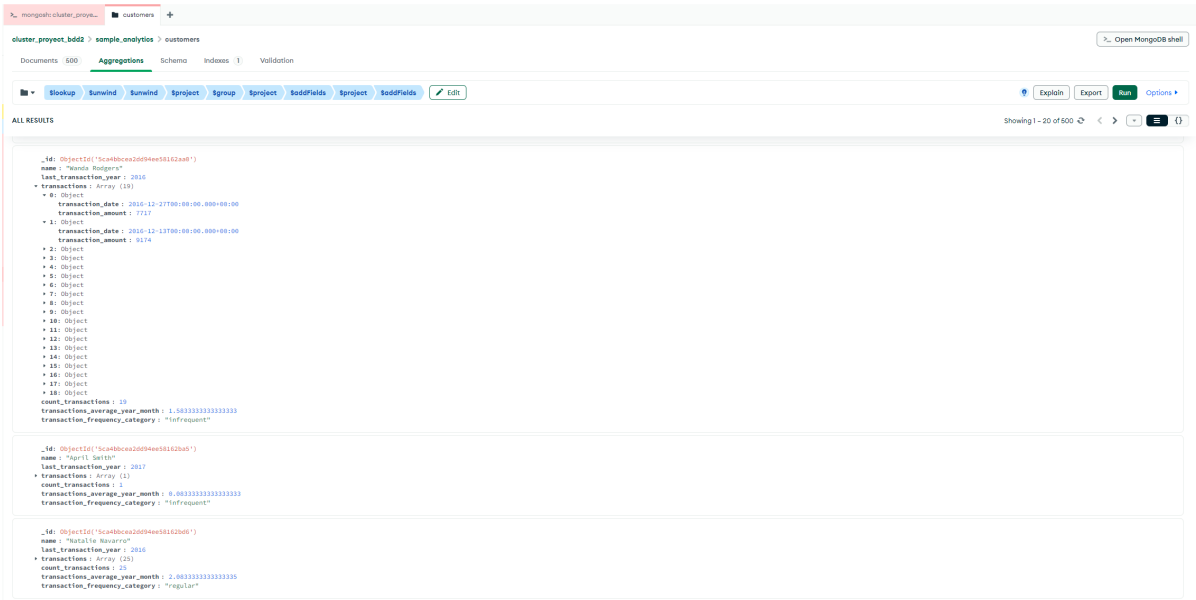


Figure 15: Resultado Ejecución Pipeline